

# Debug JSF lifecycle

<http://balusc.blogspot.com/2006/09/debug-jsf-lifecycle.html#AddImmediateTrueToUIInputOnly>

By BalusC or Bauke Scholtz

## Listen and debug JSF lifecycle phases

The JSF lifecycle will be explained and debugged here using the "poor man's debugging" approach with `sysout`'s. We'll also check what happens if you add `immediate="true"` to the `UIInput` and `UICommand` and what happens when a `ConverterException` and `ValidatorException` will be thrown.

Well, you probably already know that the JSF lifecycle contains 6 phases:

1. Restore view
2. Apply request values
3. Process validations
4. Update model values
5. Invoke application
6. Render response

You can use a `PhaseListener` to trace the phases of the JSF lifecycle and execute some processes where required. But you can also use a "dummy" `PhaseListener` to debug the phases to see what is happening in which phase. Here is a basic example of such a **LifeCycleListener**:

*Note: if you don't have a JSF playground environment setup yet, then you may find this tutorial useful as well: [JSF tutorial with Eclipse and Tomcat](#).*

```
package mypackage;

import javax.faces.event.PhaseEvent;
import javax.faces.event.PhaseId;
import javax.faces.event.PhaseListener;

public class LifeCycleListener implements PhaseListener {

    public PhaseId getPhaseId() {
        return PhaseId.ANY_PHASE;
    }

    public void beforePhase(PhaseEvent event) {
        System.out.println("START PHASE " + event.getPhaseId());
    }

    public void afterPhase(PhaseEvent event) {
        System.out.println("END PHASE " + event.getPhaseId());
    }

}
```

Add the following lines to the **faces-config.xml** to activate the LifecycleListener.

```
<lifecycle>
    <phase-listener>mypackage.LifecycleListener</phase-listener>
</lifecycle>
```

This produces like the following in the system output:

```
START PHASE RESTORE_VIEW 1
END PHASE RESTORE_VIEW 1
START PHASE APPLY_REQUEST_VALUES 2
END PHASE APPLY_REQUEST_VALUES 2
START PHASE PROCESS_VALIDATIONS 3
END PHASE PROCESS_VALIDATIONS 3
START PHASE UPDATE_MODEL_VALUES 4
END PHASE UPDATE_MODEL_VALUES 4
START PHASE INVOKE_APPLICATION 5
END PHASE INVOKE_APPLICATION 5
START PHASE RENDER_RESPONSE 6
END PHASE RENDER_RESPONSE 6
```

[Back to top](#)

### [Basic debug example](#)

To trace all phases of the JSF lifecycle, here is some sample code which represents simple JSF form with a "dummy" converter and validator and the appropriate backing bean with all component bindings. The code sample can be used to give us more insights into the phases of the JSF lifecycle, to understand it and to learn about it. **Please note that component bindings are rarely used in real world this way. They are in this code sample just there to track and log component set/get actions by JSF. In real world code you should not have the need for them.**

Here's the form, just put it in its entirety in the body of your **test.jsp** file (note: at time of writing, JSF 1.2 is used, but if you're using JSF 2.x, you can of course also put this in a **test.xhtml** file):

```
<h:form>
    <h:inputText
        binding="#{myBean.inputComponent}"
        value="#{myBean.inputValue}"
        valueChangeListener="#{myBean.inputChanged}">
        <f:converter converterId="myConverter" />
        <f:validator validatorId="myValidator" />
    </h:inputText>
    <h:commandButton
        value="submit"
        action="#{myBean.action}" />
    <h:outputText
        binding="#{myBean.outputComponent}"
        value="#{myBean.outputValue}" />
</h:form>
```

```
<h:messages />
</h:form>
```

### Here's the dummy converter: **MyConverter.java**

```
package mypackage;

import javax.faces.component.UIComponent;
import javax.faces.context.FacesContext;
import javax.faces.convert.Converter;

public class MyConverter implements Converter {

    public Object getAsObject(FacesContext context, UIComponent component,
String value) {
        System.out.println("MyConverter getAsObject: " + value);
        return value;
    }

    public String getAsString(FacesContext context, UIComponent component,
Object value) {
        System.out.println("MyConverter getAsString: " + value);
        return (String) value;
    }

}
```

### Here's the dummy validator: **MyValidator.java**

```
package mypackage;

import javax.faces.component.UIComponent;
import javax.faces.context.FacesContext;
import javax.faces.validator.Validator;
import javax.faces.validator.ValidatorException;

public class MyValidator implements Validator {

    public void validate(FacesContext context, UIComponent component, Object
value)
        throws ValidatorException
    {
        System.out.println("MyValidator validate: " + value);
    }

}
```

### Here's the backing bean: **MyBean.java**

```
package mypackage;

import javax.faces.component.html.HtmlInputText;
import javax.faces.component.html.HtmlOutputText;
import javax.faces.event.ValueChangeEvent;
```

```

public class MyBean {

    // Init -----
    -----

    private HtmlInputText inputComponent;
    private String inputValue;
    private HtmlOutputText outputComponent;
    private String outputValue;

    // Constructors -----
    -----

    public MyBean() {
        log("constructed");
    }

    // Actions -----
    -----

    public void action() {
        outputValue = inputValue;
        log("succes");
    }

    // Getters -----
    -----

    public HtmlInputText getInputComponent() {
        log(inputComponent);
        return inputComponent;
    }

    public String getInputValue() {
        log(inputValue);
        return inputValue;
    }

    public HtmlOutputText getOutputComponent() {
        log(outputComponent);
        return outputComponent;
    }

    public String getOutputValue() {
        log(outputValue);
        return outputValue;
    }

    // Setters -----
    -----

    public void setInputComponent(HtmlInputText inputComponent) {
        log(inputComponent);
        this.inputComponent = inputComponent;
    }

    public void setInputValue(String inputValue) {

```

```

        log(inputValue);
        this.inputValue = inputValue;
    }

    public void setOutputComponent(HtmlOutputText outputComponent) {
        log(outputComponent);
        this.outputComponent = outputComponent;
    }

    // Listeners -----
    -----

    public void inputChanged(ValueChangeEvent event) {
        log(event.getOldValue() + " to " + event.getNewValue());
    }

    // Helpers -----
    -----

    private void log(Object object) {
        String methodName =
Thread.currentThread().getStackTrace()[2].getMethodName();
        System.out.println("MyBean " + methodName + ": " + object);
    }
}

```

### The minimal faces configuration: **faces-config.xml**

```

<converter>
    <converter-id>myConverter</converter-id>
    <converter-class>mypackage.MyConverter</converter-class>
</converter>
<validator>
    <validator-id>myValidator</validator-id>
    <validator-class>mypackage.MyValidator</validator-class>
</validator>
<managed-bean>
    <managed-bean-name>myBean</managed-bean-name>
    <managed-bean-class>mypackage.MyBean</managed-bean-class>
    <managed-bean-scope>request</managed-bean-scope>
</managed-bean>

```

Note: at the time of writing, JSF 1.2 was the standard and JSF 2.0 didn't exist yet. If you're however using JSF 2.x, then you could of course also instead use the `@FacesConverter("myConverter")`, `@FacesValidator("myValidator")` and `@ManagedBean` `@RequestScoped` annotations respectively.

[Back to top](#)

### The first call

The first call in a freshly started webapplication with a fresh session should output at least:

```
START PHASE RESTORE_VIEW 1
END PHASE RESTORE_VIEW 1
START PHASE RENDER_RESPONSE 6
MyBean <init>: constructed
MyBean getInputComponent: null
MyBean setInputComponent: javax.faces.component.html.HtmlInputText@2a9fca57
MyBean getInputValue: null
MyBean getOutputComponent: null
MyBean setOutputComponent: javax.faces.component.html.HtmlOutputText@13bbca56
MyBean getOutputValue: null
END PHASE RENDER_RESPONSE 6
```

### **1. Restore view.**

As the session is fresh, there's no means of any UIViewRoot to restore, so nothing to see here.

### **2. Apply request values.**

This phase is skipped because there is no form submit.

### **3. Process validations.**

This phase is skipped because there is no form submit.

### **4. Update model values.**

This phase is skipped because there is no form submit.

### **5. Invoke application.**

This phase is skipped because there is no form submit.

### **6. Render response.**

The bean is constructed. Behind the scenes a new UIViewRoot is created and stored in the session. If the component binding getters returns precreated components (precreated in e.g. the constructor) and not null, then those will be used, otherwise JSF will create new components. The components will be stored in the UIViewRoot and the bounded components are set in the component bindings. The values to be shown are retrieved from the value binding getters in the backing bean. If the values aren't set yet, they defaults to null. The component bindings are not required by the way. Only use them if you actually need the component in the backing bean for other means than getting/setting the value. In this article they are included just to demonstrate what all happens in the lifecycle.

[Back to top](#)

### [The form submit](#)

The form submit with the value "test" entered should output at least:

```
START PHASE RESTORE_VIEW 1
MyBean <init>: constructed
```

```
MyBean setInputComponent: javax.faces.component.html.HtmlInputText@2a9fca57
MyBean setOutputComponent: javax.faces.component.html.HtmlOutputText@13bbca56
END PHASE RESTORE_VIEW 1
START PHASE APPLY_REQUEST_VALUES 2
END PHASE APPLY_REQUEST_VALUES 2
START PHASE PROCESS_VALIDATIONS 3
MyConverter getAsObject: test
MyValidator validate: test
MyBean getInputValue: null
MyBean inputChanged: null to test
END PHASE PROCESS_VALIDATIONS 3
START PHASE UPDATE_MODEL_VALUES 4
MyBean setInputValue: test
END PHASE UPDATE_MODEL_VALUES 4
START PHASE INVOKE_APPLICATION 5
MyBean action: succes
END PHASE INVOKE_APPLICATION 5
START PHASE RENDER_RESPONSE 6
MyBean getInputValue: test
MyConverter getAsString: test
MyBean getOutputValue: test
END PHASE RENDER_RESPONSE 6
```

### **1. Restore view.**

The bean is constructed. The UIViewRoot is restored from session and the bounded components are set in the component bindings.

### **2. Apply request values.**

Nothing to see here. Behind the scenes the submitted form values are obtained as request parameters and set in the relevant components in the UIViewRoot, for example `inputComponent.setSubmittedValue("test")`.

### **3. Process validations.**

The submitted values are passed through the converter `getAsObject()` method and validated by the validator. If the conversion and validation succeeds, then the initial input value will be retrieved from the value binding getter and behind the scenes the `inputComponent.setValue(submittedValue)` and `inputComponent.setSubmittedValue(null)` will be executed. If the retrieved initial input value differs from the submitted value, then the `valueChangeListener` method will be invoked.

### **4. Update model values.**

The converted and validated values will now be set in the value binding setters of the backing bean. E.g. `myBean.setInputValue(inputComponent.getValue())`.

### **5. Invoke application.**

The real processing of the form submission happens here.

## 6. Render response.

The values to be shown are retrieved from the value binding getters in the backing bean. If a converter is defined, then the value will be passed through the converter `getAsString()` method and the result will be shown in the form.

[Back to top](#)

### Add immediate="true" to UIInput only

Extend the `h:inputText` in the **test.jsp** with `immediate="true"`:

```
...
<h:inputText
    binding="#{myBean.inputComponent}"
    value="#{myBean.inputValue}"
    valueChangeListener="#{myBean.inputChanged}"
    immediate="true">
...

```

The form submit with the value "test" entered should output at least:

```
START PHASE RESTORE_VIEW 1
MyBean <init>: constructed
MyBean setInputComponent: javax.faces.component.html.HtmlInputText@2a9fca57
MyBean setOutputComponent: javax.faces.component.html.HtmlOutputText@13bbca56
END PHASE RESTORE_VIEW 1
START PHASE APPLY_REQUEST_VALUES 2
MyConverter getObject: test
MyValidator validate: test
MyBean getInputValue: null
MyBean inputChanged: null to test
END PHASE APPLY_REQUEST_VALUES 2
START PHASE PROCESS_VALIDATIONS 3
END PHASE PROCESS_VALIDATIONS 3
START PHASE UPDATE_MODEL_VALUES 4
MyBean setInputValue: test
END PHASE UPDATE_MODEL_VALUES 4
START PHASE INVOKE_APPLICATION 5
MyBean action: succes
END PHASE INVOKE_APPLICATION 5
START PHASE RENDER_RESPONSE 6
MyBean getInputValue: test
MyConverter getAsString: test
MyBean getOutputValue: test
END PHASE RENDER_RESPONSE 6
```

### 1. Restore view.

The bean is constructed. The UIViewRoot is restored from session and the bounded components are set in the component bindings.

### 2. Apply request values.

Behind the scenes the submitted form values are obtained as request parameters and set in the relevant components in the UIViewRoot, for example `inputComponent.setSubmittedValue("test")`. The submitted values are immediately passed through the converter `getAsObject()` method and validated by the validator. If the conversion and validation succeeds, then the initial input value will be retrieved from the value binding getter and behind the scenes the `inputComponent.setValue(submittedValue)` and `inputComponent.setSubmittedValue(null)` will be executed. If the retrieved initial input value differs from the submitted value, then the `valueChangeListener` method will be invoked. This all happens in this phase instead of the Process validations phase due to the `immediate="true"` in the `h:inputText`.

### 3. Process validations.

Nothing to see here. The conversion and validation is already processed in the Apply request values phase, before the values being put in the components. This is due to the `immediate="true"` in the `h:inputText`.

### 4. Update model values.

The converted and validated values will now be set in the value binding setters of the backing bean. E.g. `myBean.setInputValue(inputComponent.getValue())`.

### 5. Invoke application.

The real processing of the form submission happens here.

### 6. Render response.

The values to be shown are retrieved from the value binding getters in the backing bean. If a converter is defined, then the value will be passed through the converter `getString()` method and the result will be shown in the form.

**Note for other components without `immediate`:** any other UIInput components inside the same form which don't have `immediate="true"` set will just continue the lifecycle [as usual](#).

[Back to top](#)

### [Add `immediate="true"` to UICommand only](#)

Extend the `h:commandButton` in the **test.jsp** with `immediate="true"` (don't forget to remove from `h:inputText`):

```
...
<h:commandButton
    value="submit"
    action="#{myBean.action}"
```

```
        immediate="true" />
    ...
```

The form submit with the value "test" entered should output at least:

```
START PHASE RESTORE_VIEW 1
MyBean <init>: constructed
MyBean setInputComponent: javax.faces.component.html.HtmlInputText@2a9fca57
MyBean setOutputComponent: javax.faces.component.html.HtmlOutputText@13bbca56
END PHASE RESTORE_VIEW 1
START PHASE APPLY_REQUEST_VALUES 2
MyBean action: succes
END PHASE APPLY_REQUEST_VALUES 2
START PHASE RENDER_RESPONSE 6
MyBean getOutputValue: null
END PHASE RENDER_RESPONSE 6
```

### **1. Restore view.**

The bean is constructed. The UIViewRoot is restored from session and the bounded components are set in the component bindings.

### **2. Apply request values.**

The real processing of the form submission happens here. This happens in this phase instead of the Invoke application phase due to the `immediate="true"` in the `h:commandButton`. The UIInput components which don't have `immediate="true"` set will not be converted, validated nor updated, but behind the scenes the `inputComponent.setSubmittedValue(submittedValue)` will be executed before the `action()` method will be executed.

### **3. Process validations.**

This phase is skipped due to the `immediate="true"` in the `h:commandButton`.

### **4. Update model values.**

This phase is skipped due to the `immediate="true"` in the `h:commandButton`.

### **5. Invoke application.**

This phase is skipped due to the `immediate="true"` in the `h:commandButton`.

### **6. Render response.**

The values to be shown are retrieved from the value binding getters in the backing bean, except for the UIInput components which don't have `immediate="true"` set. Behind the scenes those will be retrieved from the components in the UIViewRoot, e.g. `inputComponent.getSubmittedValue()`.

**Note for all components without immediate:** as the Update model values phase is skipped, the value bindings aren't been set and the value binding getters will return null. But the values are still available as submitted value of the relevant components in the UIViewRoot. In this case you can retrieve the non-converted and non-validated input value from

inputComponent.getSubmittedValue() in the action() method. You could even change it using inputComponent.setSubmittedValue(new Value) in the action() method.

[Back to top](#)

### Add immediate="true" to UIInput and UICommand

Extend the h:inputText as well as the h:commandButton in the **test.jsp** with immediate="true":

```
...
<h:inputText
    binding="#{myBean.inputComponent}"
    value="#{myBean.inputValue}"
    valueChangeListener="#{myBean.inputChanged}"
    immediate="true">
...
<h:commandButton
    value="submit"
    action="#{myBean.action}"
    immediate="true" />
...
```

The form submit with the value "test" entered should output at least:

```
START PHASE RESTORE_VIEW 1
MyBean <init>: constructed
MyBean setInputComponent: javax.faces.component.html.HtmlInputText@2a9fca57
MyBean setOutputComponent: javax.faces.component.html.HtmlOutputText@13bbca56
END PHASE RESTORE_VIEW 1
START PHASE APPLY_REQUEST_VALUES 2
MyConverter getAsObject: test
MyValidator validate: test
MyBean getInputValue: null
MyBean inputChanged: null to test
MyBean action: succes
END PHASE APPLY_REQUEST_VALUES 2
START PHASE RENDER_RESPONSE 6
MyConverter getAsString: test
MyBean getOutputValue: null
END PHASE RENDER_RESPONSE 6
```

#### **1. Restore view.**

The bean is constructed. The UIViewRoot is restored from session and the bounded components are set in the component bindings.

#### **2. Apply request values.**

Behind the scenes the submitted form values are obtained as request parameters and set in the relevant components in the UIViewRoot, for example inputComponent.setSubmittedValue("test"). The submitted values are immediately passed

through the converter `getAsObject()` method and validated by the validator. If the conversion and validation succeeds, then the initial input value will be retrieved from the value binding getter and behind the scenes the `inputComponent.setValue(submittedValue)` and `inputComponent.setSubmittedValue(null)` will be executed. If the retrieved initial input value differs from the submitted value, then the `valueChangeListener` method will be invoked. This all happens in this phase instead of the Process validations phase due to the `immediate="true"` in the `h:inputText`. Finally the real processing of the form submission happens here. This happens in this phase instead of the Invoke application phase due to the `immediate="true"` in the `h:commandButton`.

### 3. Process validations.

This phase is skipped due to the `immediate="true"` in the `h:commandButton`.

### 4. Update model values.

This phase is skipped due to the `immediate="true"` in the `h:commandButton`.

### 5. Invoke application.

This phase is skipped due to the `immediate="true"` in the `h:commandButton`.

### 6. Render response.

The values to be shown are retrieved from the value binding getters in the backing bean. If a converter is defined, then the value will be passed through the converter `getAsString()` method and the result will be shown in the form.

**Note for all components with immediate:** as the Update model values phase is skipped, the value bindings aren't been set and the value binding getters will return null. But the values are still available by the relevant components in the `UIViewRoot`. In this case you can retrieve the input value from `inputComponent.getValue()` in the `action()` method. The new input value is also available by the `ValueChangeEvent` in the `inputChanged()` method. You could even change it using `inputComponent.setValue(new Value)` in the `action()` method.

**Note for other components without immediate:** any other `UIInput` components inside the same form which don't have `immediate="true"` set will not be converted, validated nor updated, but behind the scenes the `inputComponent.setSubmittedValue(submittedValue)` will be executed before the `action()` method will be executed. You can retrieve the non-converted and non-validated input value from `inputComponent.getSubmittedValue()` in the `action()` method. You could even change it using `inputComponent.setSubmittedValue(new Value)` in the `action()` method.

[Back to top](#)

## Conversion error

Let's see what happens if a conversion error will occur. Change the `getAsObject()` method of **MyConverter.java** as follows (and remove the `immediate="true"` from the `test.jsp` file):

```
package mypackage;
```

```

import javax.faces.component.UIComponent;
import javax.faces.context.FacesContext;
import javax.faces.convert.Converter;
import javax.faces.convert.ConverterException;

public class MyConverter implements Converter {

    public Object getAsObject(FacesContext context, UIComponent component,
String value) {
        System.out.println("MyConverter getAsObject: " + value);
        throw new ConverterException("Conversion failed.");
    }

    ...
}

```

The form submit with the value "test" entered should output at least:

```

START PHASE RESTORE_VIEW 1
MyBean <init>: constructed
MyBean setInputComponent: javax.faces.component.html.HtmlInputText@2a9fca57
MyBean setOutputComponent: javax.faces.component.html.HtmlOutputText@13bbca56
END PHASE RESTORE_VIEW 1
START PHASE APPLY_REQUEST_VALUES 2
END PHASE APPLY_REQUEST_VALUES 2
START PHASE PROCESS_VALIDATIONS 3
MyConverter getAsObject: test
END PHASE PROCESS_VALIDATIONS 3
START PHASE RENDER_RESPONSE 6
MyBean getOutputValue: null
END PHASE RENDER_RESPONSE 6

```

### 1. Restore view.

The bean is constructed. The UIViewRoot is restored from session and the bounded components are set in the component bindings.

### 2. Apply request values.

Nothing to see here. Behind the scenes the submitted form values are obtained as request parameters and set in the relevant components in the UIViewRoot, for example `inputComponent.setSubmittedValue("test")`.

### 3. Process validations.

The submitted values are passed through the converter `getAsObject()` method, where a `ConverterException` is thrown. The validator and the `valueChangeListener` are bypassed. Also the `inputComponent.setValue(submittedValue)` won't take place. The lifecycle will proceed to the Render response phase immediately.

#### 4. Update model values.

This phase is skipped due to the ConverterException.

#### 5. Invoke application.

This phase is skipped due to the ConverterException.

#### 6. Render response.

The values to be shown are retrieved from the value binding getters in the backing bean, expecting the values for which a ConverterException has occurred. Behind the scenes those will be retrieved from the components in the UIViewRoot, e.g. `inputComponent.getSubmittedValue()`.

**Note:** any other UIInput components inside the same form which didn't throw a ConverterException will also skip the update model values and invoke application phases.

[Back to top](#)

### Validation error

Let's see what happens if a validation error will occur. Change the `validate()` method of **MyValidator.java** as follows (and remove the `immediate="true"` from the test.jsp file and revert MyConverter.java back to original):

```
package mypackage;

import javax.faces.application.FacesMessage;
import javax.faces.component.UIComponent;
import javax.faces.context.FacesContext;
import javax.faces.validator.Validator;
import javax.faces.validator.ValidatorException;

public class MyValidator implements Validator {

    public void validate(FacesContext context, UIComponent component, Object
value)
        throws ValidatorException
    {
        System.out.println("MyValidator validate: " + value);
        throw new ValidatorException(new FacesMessage("Validation failed."));
    }

}
```

The form submit with the value "test" entered should output at least:

START PHASE RESTORE\_VIEW 1

MyBean <init>: constructed

MyBean setInputComponent: javax.faces.component.html.HtmlInputText@2a9fca57

MyBean setOutputComponent: javax.faces.component.html.HtmlOutputText@13bbca56

END PHASE RESTORE\_VIEW 1

START PHASE APPLY\_REQUEST\_VALUES 2

```
END PHASE APPLY_REQUEST_VALUES 2
START PHASE PROCESS_VALIDATIONS 3
MyConverter getAsObject: test
MyValidator validate: test
END PHASE PROCESS_VALIDATIONS 3
START PHASE RENDER_RESPONSE 6
MyBean getOutputValue: null
END PHASE RENDER_RESPONSE 6
```

### **1. Restore view.**

The bean is constructed. The UIViewRoot is restored from session and the bounded components are set in the component bindings.

### **2. Apply request values.**

Nothing to see here. Behind the scenes the submitted form values are obtained as request parameters and set in the relevant components in the UIViewRoot, for example `inputComponent.setSubmittedValue("test")`.

### **3. Process validations.**

The values are retrieved as objects from the components, passed through the converter `getAsObject()` method and validated by the validator, where a `ValidatorException` is thrown. The `valueChangeListener` is bypassed. Also the `inputComponent.setValue(submittedValue)` won't take place. The lifecycle will proceed to the Render response phase immediately.

### **4. Update model values.**

This phase is skipped due to the `ValidatorException`.

### **5. Invoke application.**

This phase is skipped due to the `ValidatorException`.

### **6. Render response.**

The values to be shown are retrieved from the value binding getters in the backing bean, except of the values for which a `ValidatorException` has occurred. Behind the scenes those will be retrieved from the components in the UIViewRoot, e.g. `inputComponent.getSubmittedValue()`.

**Note:** any other UIInput components inside the same form which didn't throw a `ValidatorException` will also skip the update model values and invoke application phases.

[Back to top](#)

### **Okay, when should I use the immediate attribute?**

If it isn't entirely clear yet, here's a summary, complete with real world use examples when they may be beneficial:

- If set in UIInput(s) only, the process validations phase will be taken place in apply request values phase instead. Use this to prioritize validation for the UIInput component(s) in question. When validation/conversion fails for any of them, the non-immediate components won't be validated/converted.
- If set in UICommand only, the apply request values phase until with update model values phases will be skipped for any of the UIInput component(s). Use this to skip the entire processing of the form. E.g. "Cancel" or "Back" button.
- If set in both UIInput and UICommand components, the apply request values phase until with update model values phases will be skipped for any of the UIInput component(s) which does not have this attribute set. Use this to skip the processing of the entire form except for certain fields (with immediate). E.g. "Password forgotten" button in a login form with a required and immediate username field and a required but non-immediate password field.